Object Oriented Programming (CSEG1044)
Unit 02 – Lecture 04 Notes
Inheritance, ‘super‘, Overriding, and Dynamic Dispatch

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **2**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**

7.1 Inheritance and types (conceptual) . . . 3

7.2 `super` usage . . . 4

7.3 Overriding and dynamic dispatch . . . 4

8 Running Project Connection **4**

9 Common Mistakes and Confusions **5**

10 Worked Example: `Shape` hierarchy **5**

11 Demo / Observation Task **5**

12 Practice Check (with brief solutions) **6**

13 Self-Study Practice (no solutions) **6**

14 Further Study **6**

15 Exit Questions (with sample answers) **7**

1 Lecture Snapshot

- **Unit focus:** Classes, Inheritance, Packages, and Interfaces
- **Main new ideas:** Inheritance and types (conceptual), `super` usage, Overriding and dynamic dispatch
- **Course thread:** Use Inheritance, 'super', Overriding, and Dynamic Dispatch to strengthen the domain model, APIs, and access rules inside Campus Resource Manager.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

2 Lecture Overview

Inheritance and polymorphism are two core tools in OOP, but they must be used carefully. This lecture focuses on:

- modeling **is-a** relationships with inheritance,
- using `super` correctly,
- overriding methods and understanding **dynamic dispatch**.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Class design**: Inheritance helps only when the parent and child already have meaningful responsibilities. Main reference: `Unit 02 / Lecture 01 slides/notes`.
- **Constructors**: The rule for super(...) depends on the constructor logic from the previous lecture. Main reference: `Unit 02 / Lecture 02 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Decide is-a (inheritance) vs has-a (composition).
- Use `super(...)` and `super.method()` correctly.
- Override methods and explain dynamic dispatch with an example.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Inheritance and types (conceptual)* → ***`super`*** *usage* → *Overriding and dynamic dispatch.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Inheritance**: a subclass extends a superclass (`extends`).
- **Superclass/Subclass**: parent/child in inheritance hierarchy.
- **Overriding**: subclass provides its own implementation of a superclass method (same signature).
- **Dynamic dispatch**: method call resolved at runtime based on the actual object type.
- **Upcasting**: treating a subclass object as a superclass reference, for example `Shapes=newCircle(...)`.
- **Composition**: has-a relationship (`Car` has an `Engine`).

7 Detailed Notes

7.1 Inheritance and types (conceptual)

Java supports:

- single inheritance of implementation (one direct superclass),
- multilevel and hierarchical inheritance patterns,
- multiple inheritance of *type* through interfaces (covered more later).

Is-a vs has-a:

- Use inheritance only when the subclass is truly a kind of superclass.
- If something is a part of another thing, prefer composition.

Examples:

- Dog is-a Animal (inheritance)
- Car has-a Engine (composition)

7.2 `super` usage

Constructor: a subclass constructor must call a superclass constructor. If you do not write `super(...)`, Java inserts `super()` automatically *only if* the superclass has a no-arg constructor.

Rule: `super(...)` must be the **first statement** in the constructor (similar to `this(...)`).

Member access: use `super.method()` to explicitly call the parent method if overridden.

7.3 Overriding and dynamic dispatch

Overriding rules (important ones):

- same method signature in subclass,
- cannot reduce visibility (`public` cannot become `protected/private`),
- use `@Override` to catch mistakes.

Dynamic dispatch: if you store a circle as `Shapes=newCircle(...)`, then calling `s.area()` still runs `Circle.area()` at runtime. This is the heart of runtime polymorphism.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

8 Running Project Connection

Use Inheritance, ‘super‘, Overriding, and Dynamic Dispatch to strengthen the domain model, APIs, and access rules inside Campus Resource Manager.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today’s idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Choosing inheritance just to reduce code repetition quickly.
- Forgetting that super(...) must run before subclass-specific initialization.
- Confusing overriding with overloading.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

10 Worked Example: `Shape` hierarchy

```java
class Shape {
    private final String color;

    Shape(String color) {
        this.color = color;
    }

    double area() { return 0.0; }
    String getColor() { return color; }
}

class Circle extends Shape {
    private final double r;

    Circle(String color, double r) {
        super(color); // superclass constructor
        this.r = r;
    }

    @Override
    double area() { return Math.PI * r * r; }
}
```

```java
public class Demo {
    public static void main(String[] args) {
        Shape s = new Circle("red", 2.0); // upcasting
        System.out.println(s.area()); // dynamic dispatch
    }
}
```

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Inheritance and types (conceptual)* once without looking at the solution first.

- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. Is-a or has-a? **(a)** Student–IDCard **(b)** Manager–Employee

   Answer: (a) has-a, (b) is-a (in a simple model).

2. What does `super(...)` do in a constructor?

   Answer: calls the superclass constructor to initialize the parent part of the object.

3. If `Shape s = new Circle(...)` and you call `s.area()`, which implementation runs?

   Answer: `Circle.area()` (runtime dispatch).

13 Self-Study Practice (no solutions)

1. Create `Rectangle` and `Triangle` subclasses of `Shape` and override `area()`.
2. Design a model using composition (has-a) and justify why inheritance is not appropriate.
3. Write one example where you call `super.method()` from an overridden method.

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. Why is inheritance sometimes risky?

Sample answer: Inheritance creates strong coupling between parent and child. If the parent changes, many subclasses may break. Using inheritance only when the relationship is truly is-a helps avoid bad designs.

Q2. What is dynamic dispatch?

Sample answer: Dynamic dispatch is when the method that executes is chosen at runtime based on the actual object type, not the reference type.

Q3. How does dynamic dispatch support extensible design?

Sample answer: Client code can depend on a parent type while the correct overridden behavior is selected at runtime.